SIMULACRO DE LA ACTIVIDAD DE COMPROBACIÓN N.° 3

Sistema con almacenamiento de datos

Parte 1: corrección de código · Parte 2: pases de bus estudiantiles

Módulo	Diseño de algoritmos · CSTI12010
Nombre de la persona estudiante	_____
Fecha	Miércoles 4 de noviembre de 2026
Modalidad	Simulacro guiado por la persona docente · misma estructura y reglas que la AC3 del jueves
Persona docente	Giovanni Antonio Coto Calderón

¿Para qué es este simulacro?

Hoy se practica la AC3 **exactamente como será el jueves**: las mismas dos partes, el mismo tipo de ejercicio, las mismas reglas de material y la misma forma de entregar. Los temas son distintos. La persona docente guía cada paso; usted programa en su computadora y al final se autoevalúa con el checklist de la AC3.

Material y recursos permitidos (se practican hoy)

- **Sí se permite:** solo su material escrito a puño y letra (sus apuntes en papel).
- **Internet: no se permite**, salvo al inicio para descargar las instrucciones y al final para subir su trabajo a GitHub y enviar el correo electrónico con la carpeta comprimida.
- **No se permiten celulares ni ninguna otra fuente:** material impreso, archivos o programas guardados en la computadora o en memorias USB, páginas web, inteligencia artificial ni ayuda de otras personas.

Estructura de la actividad

- **Parte 1 · Corrección de código.** Dos programas cortos con 2 errores cada uno: encuéntrelos, corríjalos y explíquelos en comentarios.
- **Parte 2 · Ejercicio.** Programar el control de pases de bus estudiantiles.

Parte 1 · Corrección de código

Recibe los archivos **correccion1_promedio.c** y **correccion2_ordenar.c**. Cada uno tiene **2 errores**. Para cada archivo:

- Compílelo con gcc -std=c11 -Wall -Wextra, ejecútelo y compare con el resultado esperado.

- Encuentre y corrija los 2 errores. Lea las advertencias y los errores del compilador, pero recuerde que hay errores que el compilador no avisa.
- Arriba de cada línea corregida escriba un comentario con **qué pasaba**, **por qué** y **cuál es la corrección**: // ERROR: ... // POR QUE: ... // CORRECCION: ...
- Compruebe que el programa corregido compila sin advertencias y da el resultado esperado.

Corrección 1 · Promedio de notas (correccion1_promedio.c)

El programa guarda 3 notas (80, 75 y 90) en un arreglo y calcula su promedio con la función promedio.

correccion1_promedio.c

```c
#include <stdio.h>

double promedio(const int notas[], int n) {
    int i, suma = 0;
    for (i = 0; i < n; i++)
        suma = notas[i];
    return suma / n;
}

int main(void) {
    int notas[3] = {80, 75, 90};
    printf("Promedio: %.2f\n", promedio(notas, 3));
    return 0;
}
```

Datos de prueba	No pide datos: las notas ya están en el arreglo.
Resultado esperado	Promedio: 81.67

Corrección 2 · Ordenar dos números (correccion2_ordenar.c)

El programa lee dos números y, con la función ordenar, deja el menor en x y el mayor en y.

correccion2_ordenar.c

```c
#include <stdio.h>

void ordenar(int *a, int *b) {
    int temp;
    if (*a > *b) {
        temp = a;
        *a = *b;
        *b = temp;
    }
}

int main(void) {
    int x, y;
    printf("Primer numero: ");
    scanf("%d", &x);
    printf("Segundo numero: ");
    scanf("%d", &y);
    ordenar(x, y);
    printf("Menor: %d  Mayor: %d\n", x, y);
    return 0;
}
```

Datos de prueba	9 y 4; luego 3 y 7 (dos ejecuciones)
Resultado esperado	Menor: 4 Mayor: 9 · Menor: 3 Mayor: 7

Parte 2 · Caso: pases de bus estudiantiles

Un colegio controla los **pases de bus** de sus estudiantes y puede registrar **hasta 5 pases**. De cada pase se guarda: el **código** (100 a 999, no se repite), el **nombre** del estudiante (una sola palabra, hasta 19 letras), los **viajes disponibles** y si el pase está **activo** o dado de baja.

El programa inicia con **3 pases ya cargados** (tabla de abajo). Debe mostrar un **menú** que se repite hasta elegir 0; toda opción o dato fuera de rango se vuelve a pedir.

Opción del menú	Qué debe hacer
1 Incluir pase	Si hay espacio, pide código, nombre y viajes (1 a 40). Si el código ya existe en un pase activo, avisa y no lo incluye. Un pase nuevo inicia activo.
2 Consultar pases	Muestra en columnas el código, el nombre y los viajes de cada pase activo, y al final cuántos pases activos hay.
3 Registrar un viaje	Pide el código. Si le quedan viajes, resta uno y muestra cuántos quedan; si no le quedan, avisa.
4 Dar de baja un pase	Pide el código y marca el pase como dado de baja: ya no aparece en consultas ni búsquedas.
5 Buscar pase	Pide el código y muestra el código, el nombre y los viajes del pase.
0 Salir	Muestra un mensaje de despedida y termina.

En las opciones 3, 4 y 5, si el código no corresponde a un pase activo, el programa avisa «No existe un pase activo con el código …» y vuelve al menú.

Pases cargados al iniciar:

Código	Nombre	Viajes
201	Ana	5
202	Luis	1
203	Sofia	8

Requisitos técnicos

- Constantes (#define) para los límites del enunciado.
- **Arreglos paralelos** para código, nombre, viajes y estado (1 activo, 0 dado de baja), y una variable con la cantidad de posiciones usadas.
- Menú con **do-while** y **switch**; validación de todos los datos numéricos.
- Programa **modular** con prototipos antes de main y, como mínimo, las funciones de la tabla. Las marcadas deben recibir la **dirección** del dato que cambian.
- Encabezado de comentarios (enunciado, qué se logra, cómo se logra, resultado esperado) y comentarios que expliquen qué hace cada parte, cómo y para qué.

Función	Qué hace	Recibe por dirección
`leerEntero`	Pide un entero entre un mínimo y un máximo y lo vuelve a pedir si está fuera de rango (do-while).	—
`buscarCodigo`	Búsqueda secuencial de un pase ACTIVO por código; devuelve la posición o -1.	—
`incluir`	Guarda el pase en la primera posición libre y aumenta la cantidad.	La cantidad de pases (int *n)
`consultar`	Muestra la tabla de pases activos y cuántos son.	—
`usarViaje`	Resta un viaje; devuelve 1 si pudo o 0 si no quedaban.	Los viajes del pase
`darDeBaja`	Pone el estado del pase en 0.	El estado del pase

Caso de prueba

Pruebe su programa con esta secuencia, en este orden. Los resultados de la derecha son los que debe obtener.

Paso	Acción	Resultado esperado
1	2 Consultar	Los 3 pases iniciales; pases activos: 3.
2	1 Incluir, código 202	El código 202 ya existe.
3	1 Incluir: 204, Mario, 45 (inválido) y luego 3	Pide de nuevo los viajes; pase incluido (4 de 5).
4	3 Registrar viaje de 202, dos veces	A Luis le quedan 0; la segunda vez: Luis no tiene viajes disponibles.
5	4 Dar de baja 201	Pase de Ana dado de baja.
6	5 Buscar 201	No existe un pase activo con el código 201.
7	5 Buscar 203	203 Sofia: 8 viajes.
8	Opción 9 (inválida) y luego 2 Consultar	Pide de nuevo la opción; muestra 202 (0), 203 (8) y 204 (3); pases activos: 3.

Entrega (igual que el jueves)

- Parte 1: **correccion1_promedio.c** y **correccion2_ordenar.c** corregidos y comentados.
- Parte 2: **transporte.c** y **transporte.exe** (gcc -std=c11 -Wall -Wextra transporte.c -o transporte.exe).
- Los cuatro archivos dentro de **Simulacro03_PrimerNombre_PrimerApellido.zip**.
- Suba el trabajo a su repositorio de GitHub y envíe por correo electrónico la carpeta comprimida a la persona docente.

Así se evalúa (el jueves)

Con el checklist de la AC3: 4 indicadores por desempeño y 12 por producto; los de archivos van N/A. Todos los críticos deben quedar en LO LOGRÓ y como máximo 2 en PENDIENTE. Hoy se autoevalúa con el checklist del simulacro.